



CHƯƠNG 2

Triển khai ứng dụng qua Docker

Triển khai và quản trị hệ thống phần mềm



Giới thiệu nội dung

Mục tiêu chương

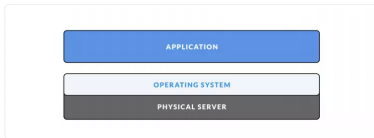
- Phân biệt VM và container; giải thích vì sao Docker phù hợp kiến trúc Microservices.
- Mô tả kiến trúc Docker và viết Dockerfile multi-stage để build Image.
- Quản lý image, container, volume, network và triển khai ứng dụng đa container.

Nội dung chính

- 1 Từ ảo hóa đến container
- 2 Kiến trúc Docker
- 3 Docker và Microservices
- 4 Dockerfile và Image
- 5 Quản lý tài nguyên Docker
- 6 Triển khai Web đa container

Bài toán “chạy tốt trên máy tôi” và máy chủ vật lý truyền thống

- Mỗi microservice có thể dùng ngôn ngữ, thư viện và phiên bản hệ thống khác nhau.
- Chuyển từ development sang testing, staging, production dễ phát sinh lỗi môi trường.
- Mô hình truyền thống: một máy chủ vật lý – một hệ điều hành – một ứng dụng.

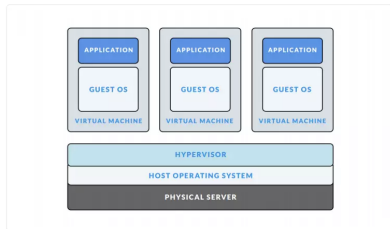


Câu hỏi mở đầu

Vì sao một ứng dụng chạy ổn trên máy lập trình viên vẫn có thể lỗi khi triển khai lên server?

Ảo hóa (Virtualization): tầng trung gian giữa phần cứng và phần mềm

- Phần cứng ngày càng mạnh nhưng mô hình 1 server/1 OS/1 app không tận dụng hết tài nguyên và khó mở rộng.
- **Ảo hóa** tạo tầng trung gian giữa phần cứng và phần mềm, phân chia một máy chủ vật lý thành nhiều máy chủ logic.
- Mỗi máy ảo (VM) có hệ điều hành riêng, ứng dụng riêng, độc lập với các máy ảo khác.



Hạn chế của máy ảo (Virtual Machine)

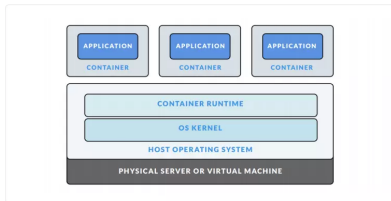
- Ảo hóa giúp một máy vật lý chạy nhiều môi trường, tăng năng suất sử dụng phần cứng.
- Nhưng việc ảo hóa cũng phát sinh vấn đề mới khi số lượng VM tăng lên.

Ba hạn chế chính của VM

- **Ngốn tài nguyên:** máy 512GB SSD / 16GB RAM tạo 2 VM (mỗi VM 128GB/4GB) sẽ mất 256GB SSD và 8GB RAM dù VM không chạy gì.
- **Tốn thời gian thực thi:** khởi động, restart, shutdown VM thường mất vài phút.
- **Cồng kênh:** khó đạt hiệu suất tối đa, khó quản lý và mở rộng.

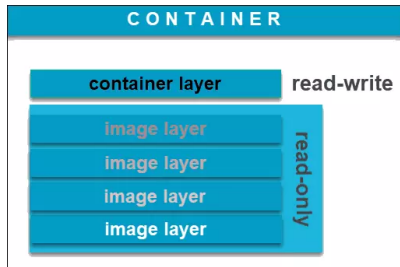
Container: chia sẻ kernel nhẹ hơn Virtual Machine

- Container cho phép một máy chủ vật lý sinh ra và chạy nhiều container, tương tự VM.
- Điểm khác biệt: các container **dùng chung kernel** và chia sẻ tài nguyên máy chủ (CPU, RAM...).
- Tài nguyên được sử dụng linh động hơn, không bị giới hạn cứng như VM.



Cấu trúc layer: vì sao container tiết kiệm lưu trữ

- Container được xây dựng từ các **image layer chỉ đọc** (read-only).
- Mỗi container có thêm một **layer read-write** riêng – mọi thay đổi của container chỉ ghi vào đây.
- Từ một vài image ban đầu có thể tạo nhiều container khác nhau → tiết kiệm không gian lưu trữ.



So sánh Container và Virtual Machine

Tiêu chí	Virtual Machine (VM)	Container (Docker)
Kích thước	Lớn (GB – vài chục GB)	Nhỏ (MB – vài trăm MB)
Khởi động	Chậm (phút)	Nhanh (giây)
Hiệu suất	Thấp hơn (emulate hardware)	Cao (chia sẻ kernel host)
Cấp độ ảo hóa	Hardware/OS	OS-level (process isolation)

Hình dung

VM như một ngôi nhà riêng biệt hoàn chỉnh. Container như các căn hộ trong chung cư – chia sẻ nền tảng (kernel) nhưng có không gian riêng.

Vì sao Containerization phù hợp với Microservices

- **Tính di động:** “Build once, run anywhere” – chạy trên laptop, on-premise, cloud mà không đổi.
- **Tính nhất quán:** môi trường giống hệt giữa dev, test, prod → giảm lỗi môi trường.
- **Hiệu suất cao:** khởi động vài giây, tiêu tốn ít tài nguyên hơn VM.
- **Dễ scale và cô lập:** mỗi container có filesystem, process riêng → tăng bảo mật.
- **CI/CD thân thiện:** dễ tích hợp Jenkins, GitHub Actions, Kubernetes.

Ví dụ

Netflix, Spotify và nhiều ngân hàng Việt Nam dùng container để quản lý hàng nghìn microservice; trong An toàn thông tin, Docker giúp triển khai nhanh, lắp lại và cô lập các công cụ bảo mật (vulnerability scanner, honeypot, SIEM agent).

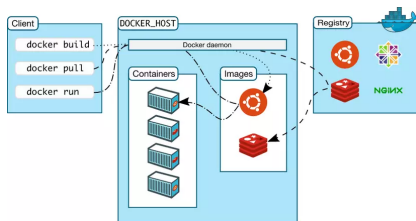
Docker: nền tảng containerization phổ biến nhất

- Đóng gói toàn bộ ứng dụng cùng dependencies (thư viện, cấu hình, runtime) vào một đơn vị chuẩn hóa gọi là **container**.
- Tiêu chí cốt lõi: “**Build once, run anywhere**”.
- Thực hiện ảo hóa ở **mức hệ điều hành**: mỗi container cô lập nhưng dùng chung bin/lib và kernel của Host OS.
- Chạy được trên Linux, Windows, macOS; hỗ trợ các dịch vụ cloud như Azure, AWS.



Docker hoạt động theo mô hình Client-Server

- Gồm Docker Engine (Daemon + CLI), Images, Containers, Registry và các thành phần hỗ trợ (Volumes, Networking).
- Dùng ảo hóa cấp hệ điều hành (Namespace/Cgroups) để cô lập ứng dụng trong container nhẹ, hiệu suất cao.
- Hai phiên bản phân phối: **Community Edition (CE)** miễn phí, và **Enterprise (EE)** có hỗ trợ cùng tính năng quản lý/bảo mật nâng cao.



Ba thành phần giao tiếp: Client, REST API, Daemon

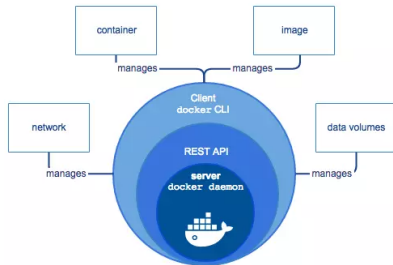
- **Docker Client (CLI):** giao diện dòng lệnh cho người dùng (VD: `docker build`, `docker run`), gửi request đến Daemon.
- **REST API:** giao diện để Client giao tiếp với Daemon.
- **Docker Daemon (`dockerd`):** “trái tim” của Docker, chạy ngầm, quản lý Image/Container/Network/Volume, nhận và thực thi request từ Client.

Điểm cốt lõi

Daemon là một lightweight runtime giúp build, run và quản lý container – người dùng chỉ tương tác gián tiếp qua Client/CLI.

Bốn đối tượng chính do Docker Daemon quản lý

- **Image:** bản mẫu chỉ đọc (read-only), chứa mã nguồn, thư viện, cấu hình; xây dựng theo layer.
- **Container:** phiên bản thực thi (instance) của Image – tiến trình cô lập, nhẹ, có thể khởi chạy/dừng/xóa.
- **Volume:** lưu trữ dữ liệu bền vững, tách biệt khỏi container.
- **Network:** cấu hình mạng giúp container giao tiếp với nhau hoặc Internet.



Vòng đời cơ bản: Build → Push/Pull → Run

- 1 **Build:** viết cấu hình trong Dockerfile, dùng Docker Client gửi lệnh build Image.
- 2 **Push/Pull:** Image được lưu vào **Registry** (VD: Docker Hub, Private Registry).
- 3 **Run:** Docker Daemon lấy Image và khởi chạy Container.

Cấu hình tối thiểu

Để chạy Docker: CPU 1–2 cores, RAM tối thiểu 1GB (khuyến nghị $\geq$ 2GB).

Kiến trúc Monolithic: một codebase duy nhất

Đặc điểm

- Mọi chức năng (auth, business logic, database, UI...) nằm trong một codebase, triển khai như một đơn vị.
- Build, test, deploy đồng thời toàn bộ ứng dụng.
- Dùng chung một cơ sở dữ liệu.

Hạn chế

- Khó mở rộng: phải sao chép toàn bộ ứng dụng.
- Thêm tính năng mới ngày càng phức tạp, khó bảo trì.
- Khó đổi công nghệ mà không viết lại toàn bộ hệ thống.

Điểm vỡ trận

Một thay đổi nhỏ ở module thanh toán có thể làm toàn bộ hệ thống downtime.

Kiến trúc Microservices: chia nhỏ thành các dịch vụ độc lập

- Mỗi service chịu trách nhiệm cho **một chức năng kinh doanh cụ thể**, có codebase, ngôn ngữ và database riêng.
- Triển khai độc lập, giao tiếp qua API nhẹ (RESTful API, gRPC, Message Queue như Kafka).
- Mỗi service chạy và scale độc lập với các service khác.

Hệ thống ngân hàng trực tuyến

User Service (quản lý tài khoản), Loan Service (xét duyệt vay), Transaction Service (chuyển tiền), Notification Service (gửi SMS/Email).

So sánh Monolithic và Microservices

Tiêu chí	Monolithic	Microservices
Cấu trúc	Một codebase lớn, tightly coupled	Nhiều service nhỏ, loosely coupled
Triển khai	Toàn bộ ứng dụng cùng lúc	Từng service độc lập
Scale	Scale toàn bộ	Scale từng service (horizontal)
Fault Isolation	Một lỗi có thể sập toàn hệ thống	Lỗi một service không ảnh hưởng toàn bộ

Lợi ích và thách thức của Microservices

Lợi ích

- Deploy nhiều lần/ngày không ảnh hưởng service khác.
- Scale linh hoạt theo từng service.
- Đa dạng công nghệ theo từng service.
- Vẫn hoạt động khi một service bị lỗi.

Thách thức

- Phức tạp phân tán: networking, service discovery, load balancing.
- Data consistency: mỗi service một database riêng.
- Overhead vận hành: monitoring, logging, security.
- Latency mạng giữa các service.

Docker – “Người bạn đồng hành hoàn hảo” của Microservices

- **Đóng gói và cô lập:** mỗi microservice thành một Image riêng → container chạy độc lập, môi trường giống hệt nhau.
- **Tính nhất quán:** giải quyết vấn đề “Works on my machine”.
- **Triển khai nhanh, scale dễ:** khởi động container chỉ trong vài giây.
- **Bảo mật tốt hơn:** cô lập bằng namespaces/cgroups, mỗi service có attack surface riêng, dễ áp dụng đặc quyền tối thiểu và quét lỗ hổng.
- **Docker Compose** định nghĩa/chạy nhiều container cùng lúc;
Kubernetes tự động hóa quản lý ở production.

Dockerfile: bản hướng dẫn xây dựng Image

- Tập văn bản chứa các **chỉ lệnh (instructions)** để Docker Daemon xây dựng Docker Image.
- Giống “bài hướng dẫn nấu ăn”: ghi từng bước, Docker thực hiện theo thứ tự để tạo ra Image.
- Mỗi chỉ lệnh tạo ra một **layer**; Image được build từ trên xuống dưới.
- Thường đặt ở thư mục gốc dự án; nên lưu trong Git để quản lý phiên bản và đảm bảo build lặp lại được (reproducible).

Các chỉ lệnh Dockerfile quan trọng nhất

Chỉ lệnh	Mô tả	Ghi chú
FROM	Chỉ định base image	Phải là lệnh đầu tiên
WORKDIR	Thiết lập thư mục làm việc	Nên dùng sớm
COPY	Copy file/thư mục từ host vào image	Tốt hơn ADD
RUN	Thực thi lệnh khi build	Tạo layer mới
EXPOSE	Khai báo port container lắng nghe	Chỉ mang tính documentation
CMD	Lệnh mặc định khi container start	Chỉ dùng 1 CMD
USER	Chạy container với user không phải root	Tăng bảo mật

Dockerfile multi-stage cho ứng dụng Node.js: Stage 1 – Build

```
# Stage 1: Build (sẽ tối ưu sau)
FROM node:20-alpine AS builder
WORKDIR /app
# Copy package files trước để tận dụng cache layer
COPY package*.json ./
RUN npm ci --only=production
# Copy source code
COPY . .
```

Layer caching

Copy `package*.json` trước rồi mới `RUN npm ci`: nếu source code đổi nhưng dependency không đổi, layer cài đặt vẫn được lấy từ cache.

Dockerfile multi-stage: Stage 2 – Production (bảo mật)

```
# Stage 2: Production
FROM node:20-alpine
WORKDIR /app
# Copy từ builder
COPY --from=builder /app /app
# Security: chạy với user không root
USER node
EXPOSE 3000
ENV NODE_ENV=production
CMD ["node", "server.js"]
```

Vì sao tách 2 stage

Chỉ Stage 2 được giữ lại làm Image production; công cụ build ở Stage 1 không có mặt, giúp Image gọn và an toàn hơn.

Tối ưu Dockerfile: base nhẹ, cache, giảm kích thước

- **Base image nhẹ:** ưu tiên `alpine`, `slim` hoặc `distroless` thay vì image đầy đủ (ubuntu, debian).
- **Tận dụng layer caching:** copy `package.json` trước khi copy toàn bộ source; đặt lệnh ít đổi (`ENV`, `WORKDIR`) lên trên, lệnh hay đổi (`COPY` source, `RUN npm install`) xuống dưới.
- **Giảm kích thước Image:** kết hợp nhiều lệnh `RUN` bằng `&&`; xóa cache/file tạm sau khi cài đặt.

Bảo mật khi xây dựng Docker Image

- Không chạy container với tài khoản root – tạo và dùng tài khoản người dùng thường (`USER`).
- Không gán cứng thông tin nhạy cảm (password, API key) trong Dockerfile → dùng `ARG/ENV` hoặc Docker Secret.
- Quét lỗ hổng Image bằng **Trivy** hoặc **Docker Scout** trước khi deploy.
- Dùng `.dockerignore` để loại trừ file không cần thiết (`node_modules`, `.git`, `*.log...`).

Ghi nhớ

Đây là các cơ chế bảo mật cơ bản nhất cần áp dụng cho mọi Image trước khi đưa vào môi trường thật.

Build Image từ Dockerfile

```
# Build với tag
$docker build -t myapp:v1.0 .
# Build không cache
$docker build --no-cache \
  -t myapp:v1.0 .
# Xem layers của image
$docker history myapp:v1.0
```



- **-t**: gắn tag để đánh dấu, phân loại phiên bản Image.
- **.**: thư mục hiện hành chứa Dockerfile và các tập tin cần copy.

Quản lý Docker Image qua CLI

Lệnh	Mô tả	Ví dụ
<code>docker images</code>	Liệt kê image hiện có	<code>-a</code>
<code>docker pull</code>	Tải image từ Registry	<code>nginx:alpine</code>
<code>docker tag</code>	Đổi tên/tag cho image	<code>-</code>
<code>docker push</code>	Đẩy image lên Registry	<code>-</code>
<code>docker rmi</code>	Xóa image	<code>-</code>
<code>docker image prune</code>	Xóa image không dùng	<code>-a --force</code>

Lưu ý

Tránh dùng tag `latest` trong production; dùng `docker system df` để xem dung lượng Image/Container/Volume đang chiếm.

Ví dụ: xây dựng, gắn tag và xóa Image

```
# Xây dựng và tag image
docker build -t webapp:1.0 .
docker tag webapp:1.0 mycompany/webapp:1.0
# Liệt kê các Image
docker images
# Xóa image cũ
docker rmi webapp:0.9
```

Quản lý Container qua CLI

Lệnh	Mô tả	Ví dụ
<code>docker run</code>	Tạo và chạy container mới	–
<code>docker ps</code>	Liệt kê container đang chạy	<code>-a</code>
<code>docker start/stop/restart</code>	Khởi động/dừng/khởi động lại	–
<code>docker rm</code>	Xóa container	<code>-f</code>
<code>docker logs</code>	Xem log container	<code>-f</code>
<code>docker exec</code>	Chạy lệnh trong container đang chạy	<code>-it ... sh</code>

Giải phẫu một lệnh `docker run` đầy đủ

```
$docker run -d --name myapi \
-p 3000:3000 \
-e NODE_ENV=production \
-v /data/logs:/app/logs \
--restart=unless-stopped \
myapi:1.2
```

- `-d`: chạy ngầm (detached mode).
- `--name`: đặt tên container.
- `-p host:container`: ánh xạ cổng.
- `-e`: truyền biến môi trường.
- `-v host:container`: mount volume.
- `--restart`: chính sách khởi động lại (`no`, `always`, `on-failure`, `unless-stopped`).

Lưu trữ dữ liệu: Volume, Bind mount, tmpfs

- Mặc định, dữ liệu ghi trong container nằm trên **writable layer** – mất khi container bị xóa.
- **Volume**: do Docker daemon quản lý hoàn toàn, bền vững, hiệu năng cao, hoạt động trên cả Linux/Windows – **ưu tiên** cho dữ liệu bền vững.
- **Bind mount**: liên kết trực tiếp một đường dẫn bất kỳ trên host – phù hợp khi cần truy cập file từ cả host và container.
- **tmpfs**: lưu trong RAM, không ghi xuống đĩa – dùng cho dữ liệu tạm/nhạy cảm (cache, credentials).

Khuyến nghị

Ưu tiên Volume cho production; dùng Bind mount khi cần chỉnh sửa file trực tiếp từ host trong lúc phát triển.

Cú pháp gắn Volume vào Container

```
$docker run --mount type=volume,src=<volume-name>,dst=<mount-path>  
$docker run --volume <volume-name>:<mount-path>
```

- Nên ưu tiên `--mount`: rõ ràng hơn, hỗ trợ đầy đủ tùy chọn (subpath, readonly, volume driver...).
- `--volume` / `-v`: 3 trường ngăn cách bởi dấu “:” – tên volume, đường dẫn mount, tùy chọn (VD: `ro`).
- Truy cập trực tiếp dữ liệu volume từ host (không qua mount) không được hỗ trợ, có thể gây hỏng dữ liệu.

Ví dụ: Volume cho cơ sở dữ liệu PostgreSQL

```
$docker run -d \  
  --name mydb \  
  -e POSTGRES_PASSWORD=Secret123 \  
  -v postgres_data:/var/lib/postgresql/data \  
  -p 5432:5432 \  
  postgres:16-alpine
```

Ví dụ

Dữ liệu CSDL được lưu trong volume `postgres_data`; khi container `mydb` bị xóa, dữ liệu vẫn được bảo toàn.

Docker Network: kết nối các container

- Hệ thống mạng ảo cho phép các container giao tiếp với nhau hoặc với dịch vụ bên ngoài.
- Docker tạo sẵn 3 network mặc định: **bridge** (mặc định), **host**, **none**.
- Cung cấp **network isolation**, **DNS resolution** tự động giữa các container, và **IPAM** tích hợp.
- Driver phổ biến khác: **overlay** (đa host, Docker Swarm), **macvlan** (gán MAC thật).

Khuyến nghị production

Nên tạo **user-defined network** riêng cho từng nhóm service thay vì dùng bridge mặc định – cô lập tốt hơn, tự động phân giải DNS.

Tạo Network và kết nối container qua tên (hostname)

```
# Tạo network
$docker network create backend-net

# Chạy database
$docker run -d --name db \
  --network backend-net \
  -e POSTGRES_PASSWORD=123 \
  postgres:16

# Chạy API kết nối với database
docker run -d --name api \
  --network backend-net \
  -p 3000:3000 \
  -e DB_HOST=db \
  myapi:1.0
```

- Container `api` truy cập được `db` qua hostname `db` mà không cần biết IP.

Khuyến nghị khi dùng Volume và Network trong production

- **Volume:** dùng Named Volume thay vì Bind Mount; backup định kỳ; không mount thư mục nhạy cảm không cần thiết.
- **Network:** đặt từng nhóm service vào network riêng (nguyên tắc đặc quyền tối thiểu); tránh network bridge mặc định cho production.
- Kết hợp firewall (Docker + UFW/cloud security group); dùng Docker Secrets để truyền thông tin nhạy cảm.

Điểm cốt lõi

Volume tách biệt dữ liệu, dễ áp dụng encryption. Network isolation giới hạn phạm vi tấn công nếu một service bị xâm nhập.

Đóng gói ứng dụng Web bằng Docker: lợi ích

- Không còn lỗi “chạy tốt trên máy tôi”.
- Triển khai nhanh chóng và lặp lại được (reproducible).
- Dễ scale và quản lý theo hướng microservices.
- Tăng cường bảo mật qua isolation và user non-root.
- Dễ tích hợp CI/CD.

Các bước đóng gói ứng dụng Web bằng Docker

- 1 Chọn **Base Image** theo ngôn ngữ: `node:20-alpine`, `python:3.12-slim`, `php:8.3-apache`...
- 2 Cài đặt dependencies (`RUN apt-get/apk add/composer install`).
- 3 Copy source code vào thư mục web root.
- 4 Cấu hình môi trường (`ENV`, file cấu hình).
- 5 Expose port và định nghĩa lệnh chạy (`CMD/ENTRYPOINT`).
- 6 Dùng **multi-stage build** nếu cần tối ưu.
- 7 Kết hợp **Docker Compose** để quản lý multi-container (Web + DB + Cache).

Kiến trúc triển khai: từ một container đến nhiều container

- **Monolith (1 container):** PHP + Apache trong cùng một container – đơn giản, phù hợp demo nhỏ.
- **Multi-container (khuyến nghị production):** Web, Database, Cache tách riêng từng container, kết nối qua Docker Network.

Ví dụ xuyên suốt

Phần tiếp theo minh họa đóng gói một ứng dụng Web PHP hiển thị/thêm sinh viên, kết nối CSDL MySQL – mô hình multi-container.

Ví dụ thực hành: ứng dụng quản lý sinh viên (PHP + MySQL)

- Hiện thị danh sách sinh viên từ CSDL MySQL.
- Thêm sinh viên mới qua form cơ bản.
- Cấu trúc thư mục dự án gồm mã nguồn PHP, Dockerfile và dữ liệu khởi tạo CSDL.

```
php-web-app/  
├─ Dockerfile  
├─ src/  
│   ├─ index.php  
│   ├─ add.php  
│   └─ config.php  
├─ data/  
│   └─ init.sql  
└─ .dockerignore
```

Dockerfile cho ứng dụng Web PHP

```
# Dockerfile
FROM php:8.3-apache
# Cài đặt các extension PHP cần thiết
RUN docker-php-ext-install pdo pdo_mysql mysqli
# Copy source code vào web root của Apache
COPY src/ /var/www/html/
# Phân quyền cho thư mục web
RUN chown -R www-data:www-data /var/www/html \
    && chmod -R 755 /var/www/html
# Bật mod rewrite (nếu sau này dùng .htaccess)
RUN a2enmod rewrite
# Expose port
EXPOSE 80
# Chạy Apache ở foreground
CMD ["apache2-foreground"]
```

Bước 1–2: Tạo Network và chạy MySQL Container

```
$docker network create php-network
```

```
$docker run -d \  
  --name mysql-db \  
  --network php-network \  
  -e MYSQL_ROOT_PASSWORD=Password123 \  
  -e MYSQL_DATABASE=university \  
  -v mysql_data:/var/lib/mysql \  
  -v $(pwd)/data/init.sql:/docker-entrypoint-initdb.d/init.sql \  
  mysql:8.0
```

Bước 3–4: Build Image và chạy PHP Web Container

```
$docker build -t php-web-app:1.0 .
```

```
$docker run -d \  
--name php-web \  
--network php-network \  
-p 8080:80 \  
php-web-app:1.0
```

- Dấu `.` trong lệnh build: Docker tìm và build Image từ file trong thư mục hiện hành.

Bước 5: Kiểm tra và Debug hệ thống nhiều container

```
# Xem danh sách container đang chạy
$docker ps
# Xem log của web
$docker logs php-web
# Xem log của database
$docker logs mysql-db
# Vào bên trong container PHP để test
$docker exec -it php-web bash
# Kiểm tra kết nối mạng
$docker exec -it php-web ping mysql-db
```

- Truy cập ứng dụng tại `http://localhost:8080`.

Tổng kết ví dụ: Dockerfile, Volume và Network phối hợp

- **Dockerfile** build Image cho ứng dụng PHP.
- **Volume** (`mysql_data`) lưu dữ liệu CSDL MySQL bền vững, không mất khi container bị xóa.
- **Network** (`php-network`) cho phép container PHP và MySQL giao tiếp qua tên container.

Điểm cốt lõi

Một hệ thống nhiều container thực tế luôn cần phối hợp cả ba thành phần này – đây là nền tảng để tiếp cận Docker Compose và Kubernetes sau này.

Tổng kết chương

Những điểm cần ghi nhớ

- Container chia sẻ kernel của host nên nhẹ và khởi động nhanh hơn máy ảo (VM).
- Docker theo mô hình Client-Server: Client gửi lệnh tới Daemon quản lý Image, Container, Volume, Network.
- Dockerfile định nghĩa Image theo lớp (layer); multi-stage build và layer caching giúp Image nhỏ, build nhanh.
- Triển khai ứng dụng thực tế cần phối hợp Dockerfile (build Image), Volume (dữ liệu bền vững) và Network (giao tiếp container).